

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	2
ОСНОВНАЯ ЧАСТЬ.....	4
1. Исследовательский раздел	4
1.1 Анализ предметной области и существующих решений.....	4
1.2 Определение требований к решению.....	7
1.3 Выбор средств разработки веб-приложения	8
1.4 Постановка задач к проектированию и разработке веб-приложения....	12
1.5 Вывод к первому разделу	12
2. Проектный раздел	13
2.1 Проектирование функциональной модели веб-приложения.....	13
2.2 Проектирование клиентской части веб-приложения	17
2.3 Проектирование серверной части веб-приложения	18
2.4 Проектирование интеграции клиентской и серверной частей	20
2.5 Проектирование схемы базы данных.....	21
2.6 Вывод ко второму разделу	22
3. Технологический раздел.....	23
3.1 Разработка модуля аутентификации пользователей	23
3.2 Вывод к третьему разделу	26
ЗАКЛЮЧЕНИЕ	27
СПИСОК ИНФОРМАЦИОННЫХ ИСТОЧНИКОВ	28

ВВЕДЕНИЕ

В современном мире информационные технологии играют важную роль в повседневной жизни человека. Различные социальные сервисы, позволяющие делиться с окружающими своими фотографиями, впечатлениями и мыслями получили широкое распространение. Во многих решениях пользователь может не только делиться различным контентом, например, фотографиями или видео, но и указывать конкретное место, в котором создан материал. Такой подход чаще всего использует технологию геотегирования [1] и открывает интересные возможности для путешественников, фотографов и блогеров, которые хотят рассказать о своих впечатлениях и показать, где происходили события, какие места они посетили и с чем столкнулись.

Для данных групп пользователей интересна не только привязка мультимедийного контента к геолокации, но и его отображение на карте [2]. Существующие сервисы часто ограничивают пользователей в интерактивности функционала. Многие популярные социальные платформы предоставляют только базовую привязку фотографий и другого мультимедийного контента к географическим координатам, не позволяя полноценно работать с картой и исследовать публикации других пользователей. Также сервисы нередко имеют ограничения в отображении контента и не предоставляют возможностей для создания пользовательских меток или фильтрации данных по типам и интересам.

Описанную проблему способно решить приложение, которое главным образом использует картографические API [3] для работы с пользовательским контентом. Такой сервис может быть представлен веб-приложением и должен реализовывать функционал по удобной и быстрой загрузке фотографий или видео, создания описания и привязке геолокации. При этом важную роль играют картографические API, которые должны обеспечивать интерактивный просмотр карты, отображение меток и фильтрацию контента.

Для выполнения практической работы необходимо решить следующие задачи:

- провести анализ предметной области, в том числе конкурентных решений;
- выбрать средства разработки веб-приложения и обосновать выбор;
- сформировать требования к разработке и поставить задачи;
- определить информационные процессы предметной области и формализовать их;
- спроектировать архитектуру веб-приложения и логику бизнес-процессов;
- спроектировать схему базы данных;
- оформить пояснительную записку согласно ГОСТ 7.32-2017 [4];
- подготовить презентацию по результатам практики.

Отчёт состоит из введения, 2 разделов, заключения, списка использованных источников.

ОСНОВНАЯ ЧАСТЬ

1. Исследовательский раздел

1.1 Анализ предметной области и существующих решений

В процессе поиска аналогичных решений выявлено, что на данный момент существует несколько сервисов для обмена геотегированным контентом и его отображением на карте, но интерактивность их функционала ограничена. В основном это выражается в том, что в качестве указываемого местоположения используется конкретный географический объект. Например, это может быть адрес здания, достопримечательность или населённый пункт. Пользователи ограничены в том, чтобы указать конкретную географическую точку, даже если в метаданных созданного контента есть информация о его точном положении. В приложениях, не использующих карту как основной объект, такой подход вполне оправдан. Однако в противном случае смысл в использовании карты как основного объекта теряется.

Также ранее существовали сервисы с полноценным интерактивным функционалом для обмена геотегированным контентом и его отображением на карте, но они претерпели значительные изменения с течением времени. Рассмотрим данные сервисы подробнее.

Panoramio [5] – сервис, основанный в 2005 году, который позволял пользователям загружать фотографии и привязывать их к географическим координатам. Загруженные фотографии отображались на картах Google Maps и Google Earth. Panoramio приобрела компания Google и впоследствии в 2016 году приняла решение о его закрытии, объяснив это стремлением перенести функции сервиса в другие свои продукты. Это вызвало недовольство среди пользователей и основателей Panoramio, которые инициировали петицию, собравшую несколько тысяч подписей [6]. Несмотря на протесты, компания Google продолжила реализацию своих планов.

Сервис предоставлял интерактивный функционал для загрузки и привязки фотографий к карте, также была возможность создавать для

фотографий описание. Впоследствии получались метки на географической карте, при клике на которые можно было просмотреть созданный контент. Пользователи могли добавлять тематические теги и категории к меткам для упрощения поиска. У каждого был свой собственный профиль с возможностью отображения всех загруженных фотографий. Таким образом, пользователь мог создать своего рода портфолио с фотографиями, личный альбом или карту воспоминаний. Сейчас от проекта остался достаточно урезанный функционал в сервисах Google Maps и Google Earth.

Яндекс.Фотки [7] – бесплатный фотохостинг, запущенный компанией Яндекс в 2007 году и закрытый в 2018 году. Сервис предоставлял пользователям аналогичный функционал. Основная цель платформы заключалась в создании удобного пространства для обмена изображениями и интеграции с другими продуктами компании, такими как Яндекс.Карты и уже закрытый сервис блогов Я.ру.

Фотографии также можно было отмечать тегами для удобного поиска. Настройки конфиденциальности позволяли делиться альбомами только с определёнными пользователями или оставлять их приватными, чего не было в сервисе Panoramio. В качестве картографической платформы использовались Яндекс.Карты. Ещё одна отличительная особенность – можно было оставлять комментарии под фотографиями и ставить лайки, что способствовало более активному развитию сообщества.

Google Earth [8] – платформа, разработанная компанией Google, которая позволяет пользователям виртуально исследовать Землю с помощью спутниковых снимков и трёхмерных моделей. Как уже отмечалось ранее, в сервис планировалось интегрировать функционал Panoramio. Фактически, всё что сейчас осталось от Panoramio – это добавление пользователями фотографий, однако данный функционал урезан. Можно добавлять по одной фотографии и привязывать её к конкретному объекту, например, к адресу. Привязка к определённой географической точке не реализована.

Сейчас Google Earth предоставляет готовый набор картографических данных и позволяет пользователям в полной мере наполнять карту. Большая часть данных создаётся компанией Google или официальными партнёрами.

Wikimapia [9] – картографический сервис, который позволяет пользователям отмечать географические объекты. Это своего рода гибрид Википедии и картографического сервиса, ориентированный на коллективное редактирование данных о географических местах. Сервис ориентирован больше на глобальную вики-карту и не предлагает пользователям использование мультимедийного контента. Также сервис предполагает коллективное редактирование и не подходит для личного использования.

На основе анализа функционала существующих решений заполнена таблица 1.1, в которой представлено сравнение основных характеристик.

Таблица 1.1 – Сравнительный анализ сервисов

Название сервиса	Panoramio	Яндекс.Фотки	Google Earth	Wikimapia
Использование мультимедийного контента	да	да	да	нет
Ориентированность на личное использование	да	да	нет	нет
Просмотр карты с отображением созданных меток	да	нет	да	да
Фильтрация контента по тегам, категориям и другим параметрам	да	да	нет	да
Возможность создавать приватный и публичный контент	нет	да	нет	нет
Возможность взаимодействовать с другими пользователями через метки (комментарии, лайки)	да	да	нет	нет

Проведённый анализ показал, что существует необходимость в создании сервиса для обмена геотегированным мультимедийным контентом, который обладал бы всеми преимуществами рассмотренных решений и устранял их недостатки. Самые приближенные решения – Panoramio и Яндекс.Фотки. В

Panoramio не было возможности создавать приватный контент, а Яндекс.Фотки предоставляли частичное отображение меток – только для групп фотографий из публикации. Данные недостатки необходимо устранить в разрабатываемом решении.

1.2 Определение требований к решению

1.2.1 Функциональные требования

Приложение должно реализовывать основные параметры, необходимые для создания интерактивной среды для обмена мультимедийным геотегированным контентом с использованием API карт. Необходимо разработать серверную часть, реализующую обработку пользовательских, географических и мультимедийных данных, а также клиентскую часть, представленную веб-сайтом. Таким образом обеспечивается кроссплатформенность решения и его интерактивная интеграция с картографическим API.

Веб-приложение должно обеспечивать следующий функционал:

- загрузка пользователем мультимедийного контента (фото, видео) и создание текстовых заметок;
- привязка контента к определённой географической точке, не зависящей от каких-либо объектов на карте;
- поддержка метаданных файлов (использование координат, записанных в метаданные файлов в момент создания);
- наличие личного профиля пользователя, где отображается созданный контент;
- отображение созданного пользователями контента на географической карте в виде меток, удобное отображение меток при изменении масштаба карты;
- создание пользователем категорий и тегов, по которым в дальнейшем можно фильтровать контент на карте;
- настройка приватности созданного контента;

– взаимодействие с другими пользователями через метки на карте (лайки и комментарии).

1.2.2 Нефункциональные требования

В качестве нефункциональных требований необходимо обеспечить следующие параметры:

- время отклика веб-приложения должно составлять не более 1 секунды при стандартной нагрузке;
- поддержка 1000 активных пользователей без деградации системы;
- автоматическое восстановление сервиса с помощью функционала Kubernetes;
- аутентификация должна реализовываться через JWT (JSON Web Token) для обеспечения будущей масштабируемости приложения;
- поддержка адаптивного дизайна для удобного отображения на различных устройствах;
- оптимизированное хранение мультимедийных данных с использованием облачных хранилищ;
- поддержка метрик для отслеживания работы приложения;
- использование Kubernetes для настройки развертывания приложения.

1.3 Выбор средств разработки веб-приложения

На основе выявленных требований определим средства для разработки веб-приложения. Важно обратить внимание на современные и надёжные технологии, обеспечивающие производительность, безопасность, отказоустойчивость, удобство разработки и сопровождения.

Для разработки серверной части веб-приложения рассматривались следующие фреймворки: Spring Framework (Spring Boot), Quarkus, Micronaut, Express.js [10].

Spring Boot обладает широкой экосистемой, стабильностью и удобным механизмом автоконфигурации. Quarkus и Micronaut лучше подходят для облачных сред, менее популярны и требуют дополнительного изучения. Express.js удобен для быстрого прототипирования, но уступает Java-

экосистеме в возможностях масштабирования, безопасности и отказоустойчивости.

Выбор сделан в пользу Spring Boot. Фреймворк предоставляет удобную автоконфигурацию и обладает богатой экосистемой. Также для обеспечения механизмов безопасности предоставляет удобный инструмент из коробки – Spring Security [11].

В основном в связке с фреймворком Spring используется язык Java, но есть альтернативные варианты – Scala, Groovy и Kotlin [12]. Java является стандартным и самым надёжным вариантом для больших приложений. Kotlin – полностью совместимый с Java язык и решает многие его проблемы, также делает код более лаконичным, читаемым и безопасным. Язык Groovy удобен для написания тестов и конфигурации, но не подходит для серверной разработки. А Scala можно использовать в высоконагруженных вычислениях, но он не является стандартным решением для Spring.

Таким образом, для разработки выбран язык Kotlin, т.к. он отлично подходит для новых проектов и решает многие архитектурные проблемы Java.

Для хранения данных приложения рассматривались СУБД MySQL, PostgreSQL и MongoDB [13]. PostgreSQL – надёжное реляционное решение, поддерживает сложные запросы и отлично подходит для работы с большим объёмом структурированных данных. MySQL подходит для простых запросов и небольших проектов, менее функционален. MongoDB – хорошее NoSQL решение, но используется преимущественно для неструктурированных данных и не поддерживает сложные реляционные связи.

Выбор сделан в пользу PostgreSQL, т.к. предполагается работа с большим объёмом структурированных данных и наличие связей.

Для хранения мультимедийного контента не подойдёт реляционная БД, также ожидается большой объём данных. В таком случае лучше всего использовать облачные хранилища, например, Amazon S3, Yandex Object Storage и Google Cloud Storage [14]. Amazon S3 и Google Cloud Storage – мощные сервисы, оптимизированные для хранения мультимедийных данных

большого размера, но сейчас на них наложены ограничения для российских пользователей. Yandex Object Storage предоставляет хороший аналог S3, интеграцию с российскими сервисами и низкую задержку запросов.

Таким образом, выбран Yandex Object Storage, как лучший вариант с учётом инфраструктурных ограничений.

Взаимодействие приложения с данными БД можно реализовать с помощью разных инструментов. Наиболее популярные в Java-приложениях – JPA, JOOQ и JDBC Template [15]. JPA использует Hibernate, предоставляет высокоуровневую абстракцию и реализует концепцию ORM, но не всегда эффективен. Часто под абстракцией скрываются лишние запросы, которые нагружают базу данных. JOOQ позволяет писать оптимизированные SQL-запросы с компиляцией на уровне кода и полностью управлять процессом доступа к данным. JDBC Template – простой инструмент, но предоставляет низкоуровневый интерфейс для запросов.

Выбор сделан в пользу JOOQ, т.к. данный инструмент уравнивает достоинства и недостатки рассмотренных решений. С одной стороны, разработчику также приходится писать SQL-запросы, но с использованием удобного DSL и компиляции, а с другой – инструмент позволяет полностью контролировать доступ к данным.

Стоит также уделить внимание изменениям схемы базы данных. Часто в процессе разработки либо при изменении требований необходимо менять структуру таблиц. Для этих целей используются инструменты миграции базы данных, такие как, например, Liquibase и Flyway [16]. Flyway подходит для простых SQL-миграций и минимальных настроек, в то время как Liquibase чаще всего используется в средних и больших проектах, где важно безопасное версионирование БД. Именно поэтому выбор пал на Liquibase как на более надёжное решение.

Для реализации клиентской части веб-приложения рассмотрим инструменты React, Vue.js и Angular [17]. React – популярное решение, но требует больше шаблонного кода и имеет сложности в настройке. Vue.js –

реактивный фреймворк, отличается гибкостью и лёгкостью в освоении, а также хорошей документацией. Инструмент Angular сложнее в освоении и работает медленнее при обновлении интерфейса.

Как фреймворк для клиентской части отлично подходит Vue.js. Также он имеет хорошую поддержку языка TypeScript, который решает основную проблему JavaScript – динамическая типизация.

Клиентская часть будет отвечать за взаимодействие с картой. Существуют разные картографические API, такие как Google Maps API, OpenStreetMap API, однако выбор был сделан в пользу Yandex Maps API [18]. Инструмент выбран как один из самых удобных и надёжных, доступных на российском рынке на данный момент. Также связка с сервисом Yandex Object Storage обеспечивает более удобную интеграцию.

Для реализации развёртки веб-приложения и мониторинга выбраны инструменты Kubernetes, Prometheus и Grafana.

Технология Kubernetes [19] выбрана как система оркестрации контейнеров, которая автоматизирует развёртывание, управление и масштабирование контейнеризированных приложений. Инструмент существенно упрощает балансировку нагрузки между экземплярами (инстансами) приложения и уменьшает количество дополнительной работы. Такие возможности не предоставляют Docker и Docker compose, т.к. инструменты ориентированы больше на работу именно с контейнерами и не предназначены для оркестрации.

Prometheus [20] – система мониторинга, которая собирает метрики с приложений и позволяет отслеживать их состояние в реальном времени. Является удобным инструментом и часто используется в Spring приложениях, также отлично подходит для сбора метрик с Kubernetes кластеров в отличие от, например, Zabbix или Datalog, сложных в настройке.

Grafana [21] – инструмент для визуализации метрик, полученных от Prometheus. Технология позволяет создавать удобные дашборды для анализа

данных приложения и отслеживания его работы. Выбран как наиболее распространённый в индустрии инструмент.

Обозначенные технологии обеспечивают гибкость, производительность и масштабируемость веб-приложения, и при правильном использовании гарантируют высокий уровень безопасности, оптимизируют процессы развёртывания, мониторинга и управления.

1.4 Постановка задач к проектированию и разработке веб-приложения

Исходя из требований поставлены задачи на проектирование и разработку веб-приложения. Определим задачи на проектирование:

- спроектировать архитектуру веб-приложения;
- спроектировать клиентскую часть;
- спроектировать серверную часть;
- спроектировать логику бизнес-процессов веб-приложения;
- спроектировать схему базы данных.

Задачи на разработку:

- разработать клиентскую часть веб-приложения;
- разработать серверную часть веб-приложения;
- интегрировать клиентскую и серверную части, реализовать схему БД;
- провести тестирование работы веб-приложения.

1.5 Вывод к первому разделу

В данном разделе изучена предметная область и функционал похожих решений, их возможности и ограничения. В ходе анализа определено, что на данный момент нет полноценных интерактивных решений для обмена геотегированным мультимедийным контентом с использованием API карт, которые покрывали бы все основные сценарии использования. Таким образом, выявлена потребность в создании веб-приложения, определены функциональные и нефункциональные требования к решению и определены инструменты для разработки. Также поставлены задачи на проектирование и реализацию веб-приложения.

2. Проектный раздел

2.1 Проектирование функциональной модели веб-приложения

Для описания процессов, происходящих в приложении, разработана функциональная схема в нотации IDEF0 (рисунки 2.1-2.6). Контекстный уровень и его декомпозиция показаны на рисунках 2.1 и 2.2 соответственно.

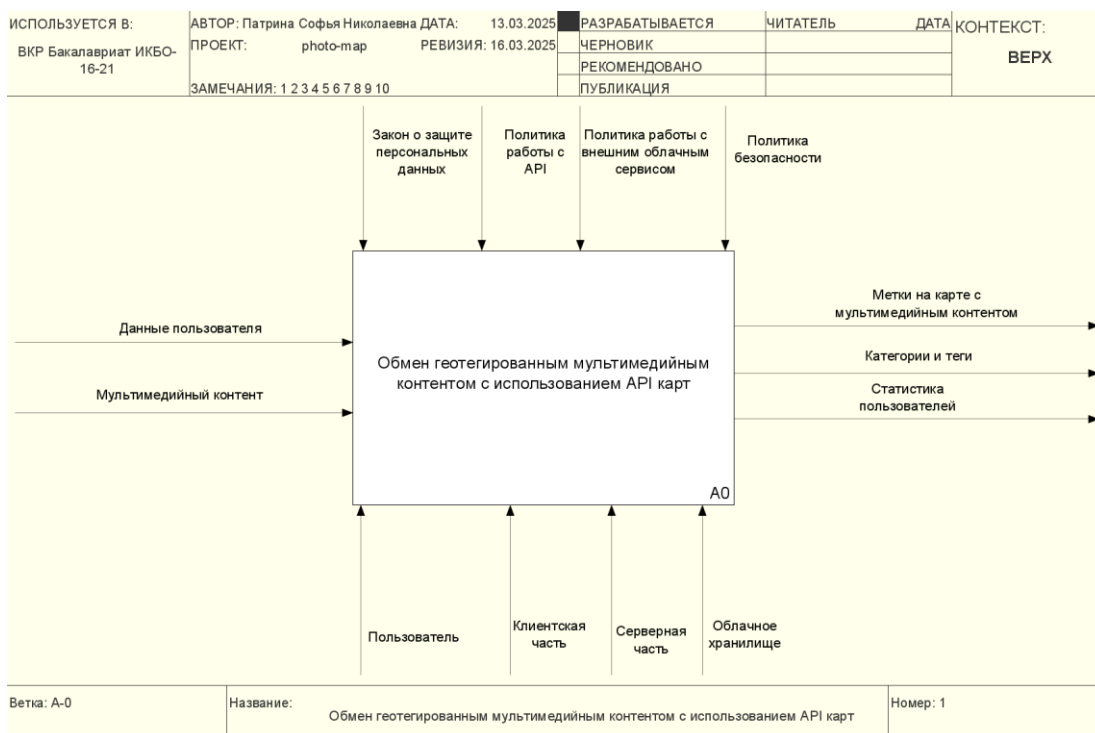


Рисунок 2.1 – Контекстный уровень функциональной схемы

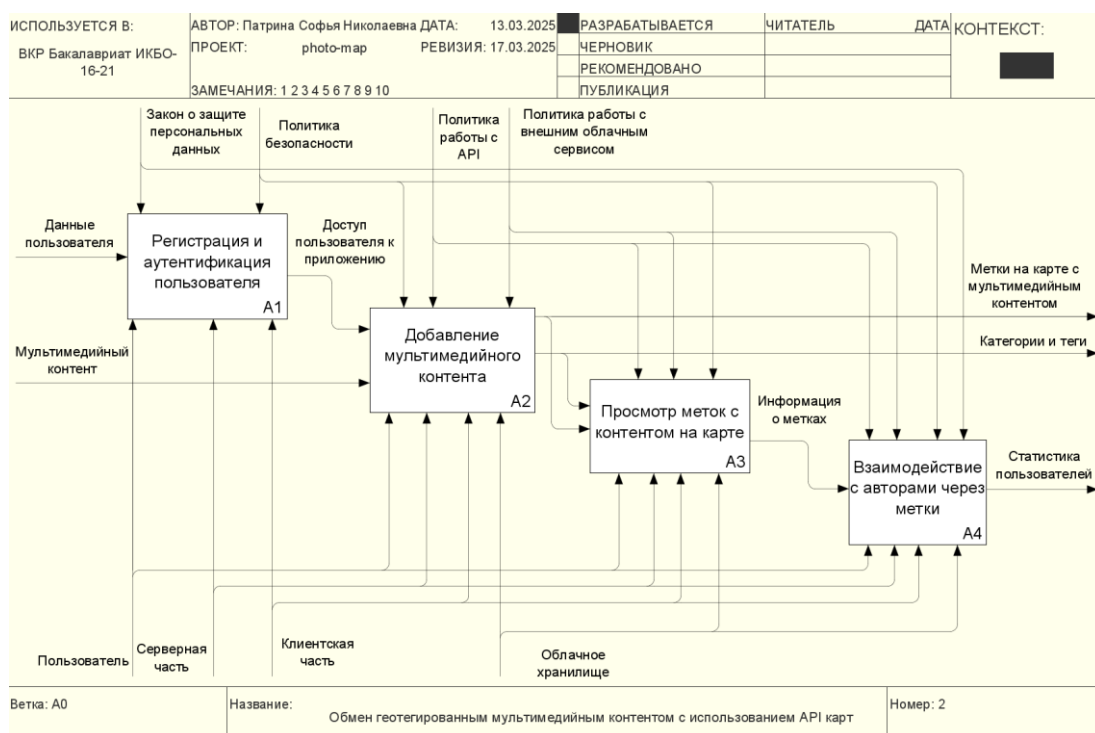


Рисунок 2.2 – Декомпозиция контекстного уровня функциональной схемы

На контекстном уровне определены входные и выходные потоки, а также потоки механизмов и управления. В качестве входных потоков определяются данные пользователя и мультимедийный контент, в качестве выходных – метки на карте с мультимедийным контентом, категории и теги, статистика пользователей. Метки на карте, категории и теги формируются из загруженного пользователем контента. Статистика пользователей формируется из его действий в приложении.

Процесс декомпозируется на регистрацию в веб-приложении, добавление мультимедийного контента на карту, просмотр созданных меток на карте и взаимодействие с пользователями, преимущественно через созданные метки.

Рассмотрим процесс регистрации пользователя. Декомпозиция данного этапа показана на рисунке 2.3.

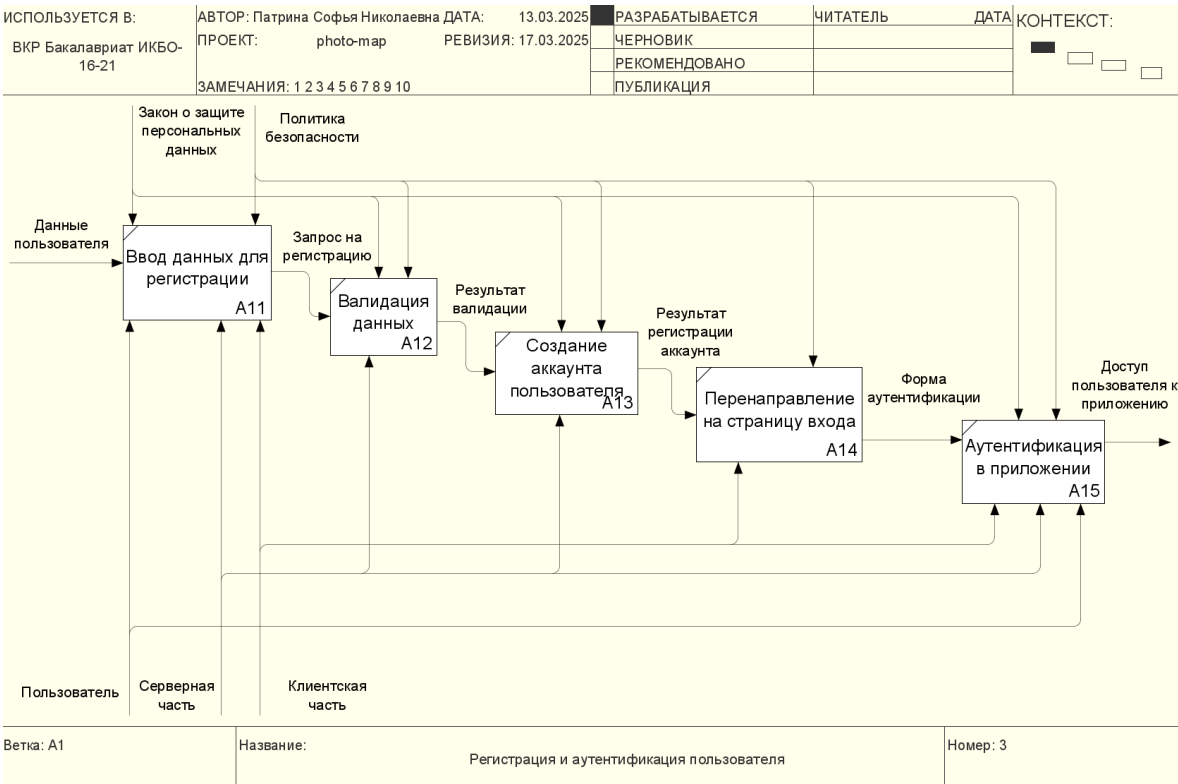


Рисунок 2.3 – Декомпозиция процесса регистрации и аутентификации

Процесс начинается с ввода данных регистрации, затем после подтверждения пользователем данные проходят валидацию и в случае успеха создаётся аккаунт. Далее пользователь перенаправляется на страницу входа, где он также вводит данные своего аккаунта, и система аутентифицирует его

в приложении. Как выходной поток данного этапа – доступ пользователя к приложению.

Далее построена декомпозиция процесса добавления мультимедийного контента (рисунок 2.4).

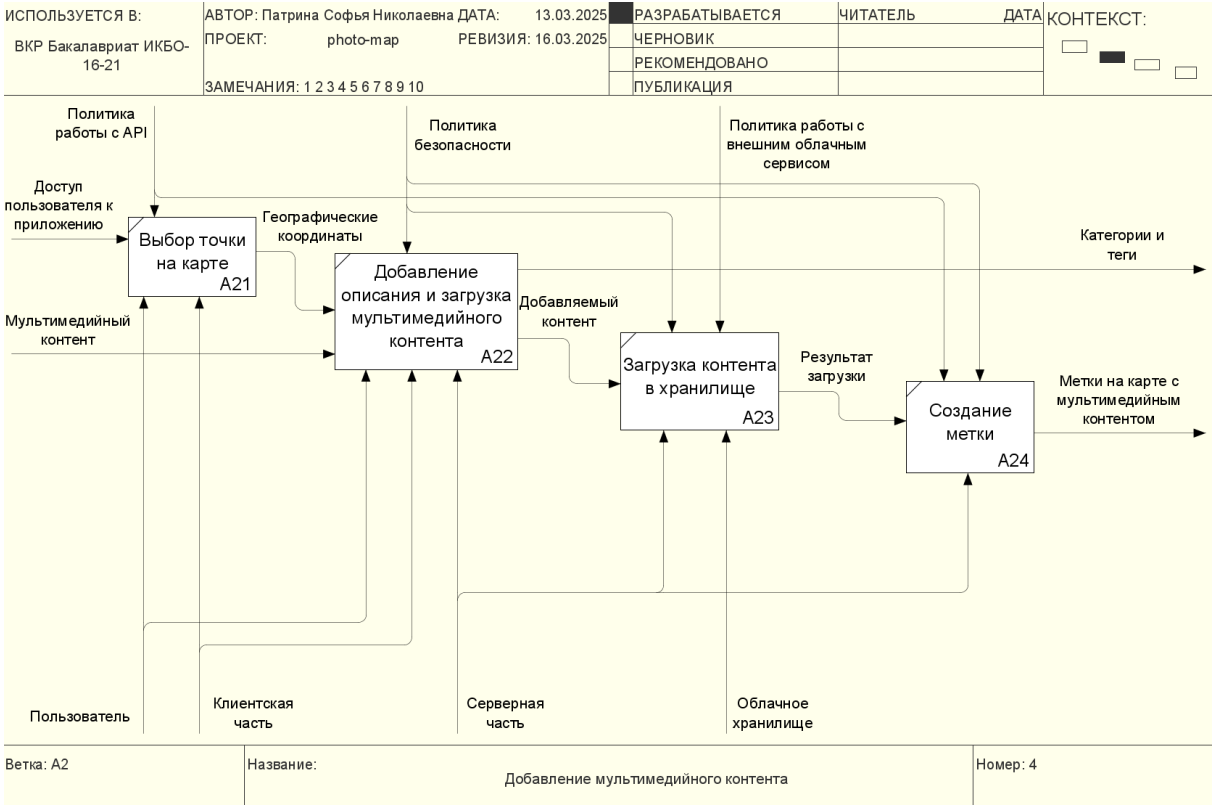


Рисунок 2.4 – Декомпозиция процесса добавления контента

Процесс состоит из четырёх основных этапов – выбор точки на географической карте, добавление описания и загрузка мультимедийного контента, загрузка контента в облачное хранилище и сохранение метки в базу данных. В качестве выходных потоков выступают категории, теги и метки с мультимедийным контентом. При создании метки пользователь может использовать уже существующие теги и категории, либо добавить новые.

Также стоит отметить, что при загрузке мультимедийного контента пользователь сможет указать более точное местоположение, если в метаданных добавляемого файла содержатся координаты.

Декомпозиции следующих процессов «Просмотр меток с контентом на карте» и «Взаимодействие с авторами через метки» показаны на рисунках 2.5-2.6.

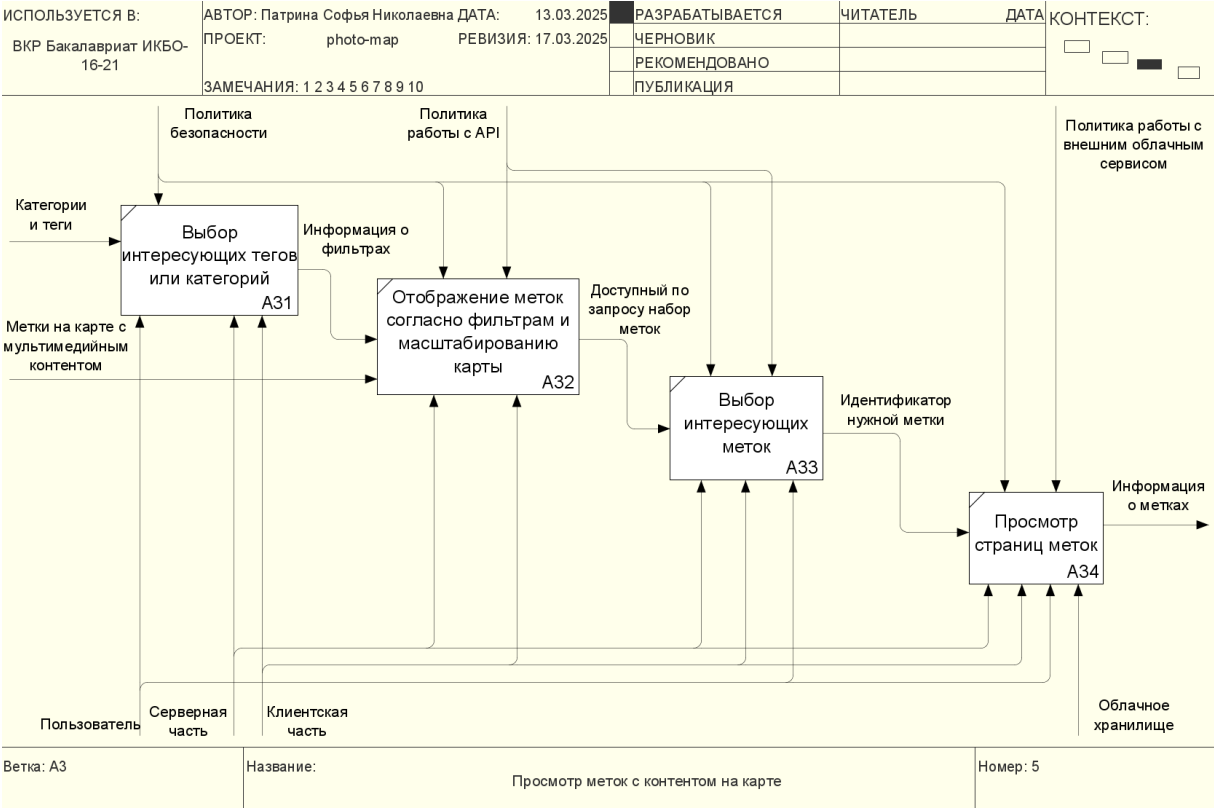


Рисунок 2.5 – Декомпозиция процесса просмотра меток на карте

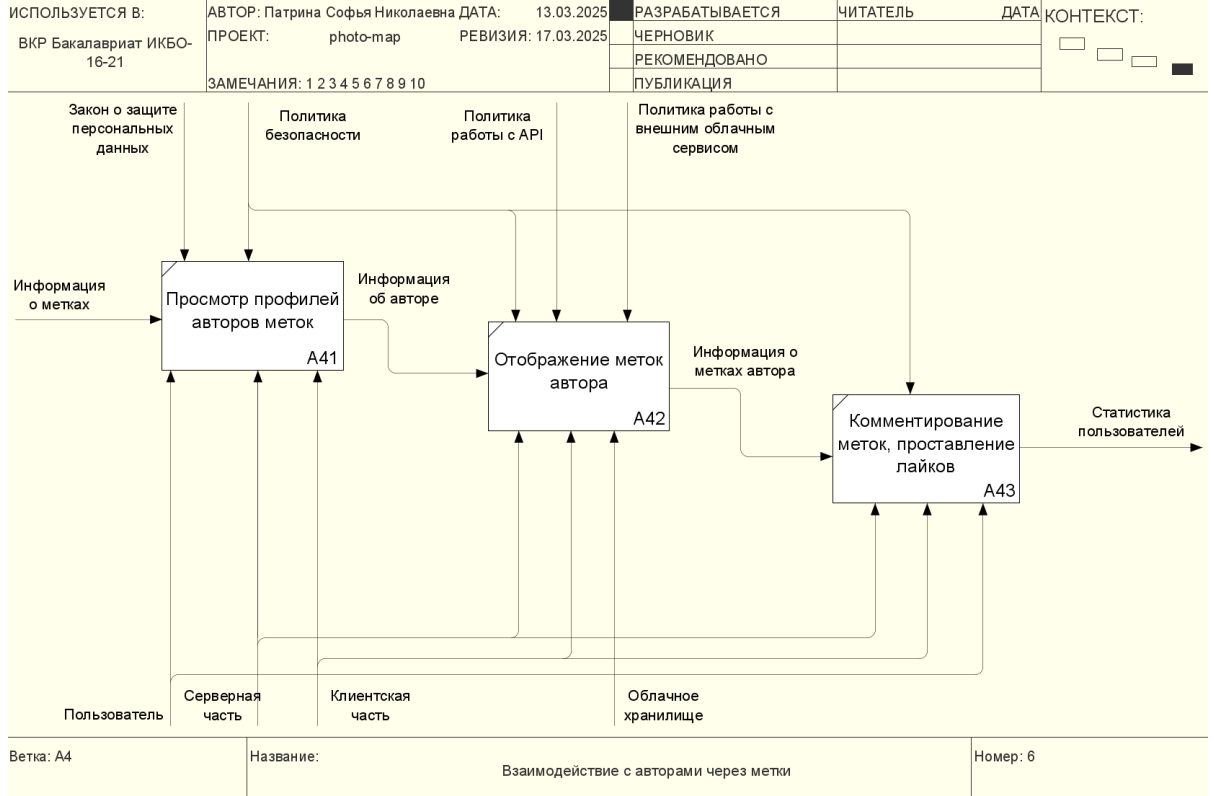


Рисунок 2.6 – Декомпозиция процесса взаимодействия с авторами

Процесс просмотра меток включает в себя 4 подпроцесса – выбор интересных тегов или категорий, отображение меток согласно фильтрам и

масштабированию карты, выбор среди них интересующих и просмотр страниц меток. При просмотре пользователь может настроить фильтры на основе интересующих тегов и категорий, чтобы сократить объём отображаемого контента. Далее пользователь может кликнуть на интересующую метку и просмотреть её описание. При переходе на страницу метки будет отображена полная информация.

Взаимодействие с авторами через метки представлено тремя подпроцессами – просмотр профилей авторов меток, отображение меток автора и комментирование, проставление лайков. При поиске меток на карте пользователь может перейти к автору и изучить его профиль. Из профиля пользователь может перейти ко всем меткам, которые создал автор, при этом они будут отображаться на карте для наглядности. На странице метки пользователь сможет проявлять свою активность в виде комментариев и лайков. Эти действия как итог формируют статистику пользователей.

2.2 Проектирование клиентской части веб-приложения

Рассмотрим варианты архитектурных подходов для клиентской части веб-приложения. Для разработки выбран фреймворк Vue.js, т.к. он обладает отличной производительностью, реактивной синхронизацией данных, имеет хорошую документацию и позволяет легко использовать известные паттерны.

В связке с Vue.js можно использовать несколько подходов. Рассмотрим архитектурный паттерн MVVM. Суть паттерна заключается в разделении компонентов на модель данных (Model), слой представления (View) и слой логики привязки данных и представления (ViewModel). Такое разделение делает разработку более удобной и позволяет изменять один слой, не затрагивая другой.

Следующий вариант для рассмотрения – компонентно-ориентированный подход [22], в котором логика разбивается на изолированные компоненты, каждый из которых отвечает за свою часть интерфейса и бизнес-логики. Подход реализуется достаточно просто – каждый компонент находится в отдельном файле и содержит в себе шаблоны, скрипты

и стили, а проект имеет ясную структуру папок, например, components, services, layouts и т.д. Такая структура упрощает поддержку проекта и дальнейшее переиспользование кода.

При стандартных сценариях разработки на Vue.js чаще всего используют паттерн MVVM и компонентно-ориентированный подход в связке. С учётом этих подходов спроектирована архитектура клиентской части веб-приложения, построена диаграмма компонентов (рисунок 2.7).

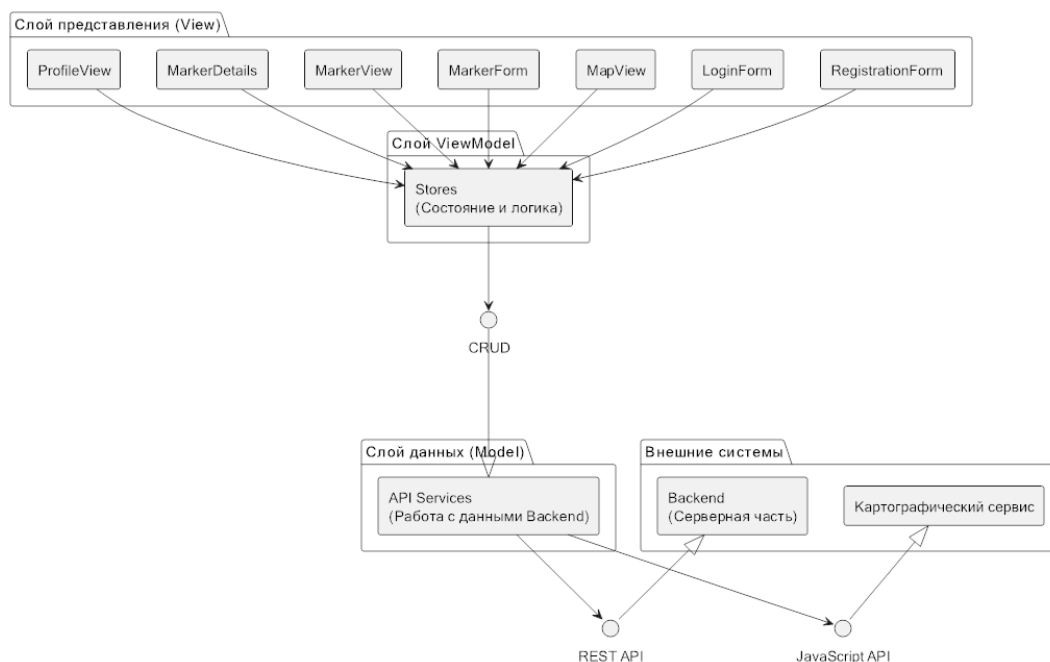


Рисунок 2.7 – Диаграмма компонентов клиентской части веб-приложения

На диаграмме в слое представления отображены основные компоненты UI, слой ViewModel содержит абстракцию stores – хранилище состояния приложения, которое используется для управления данными. Слой Model включает компоненты, которые взаимодействуют с серверной частью веб-приложения.

2.3 Проектирование серверной части веб-приложения

Серверная часть будет реализована в виде единого приложения, которое будет обеспечивать весь необходимый функционал. Данный архитектурный подход часто используется для малых и средних проектов и обеспечивает простое развёртывание и мониторинг. Серверную часть стоит разделить на модули, каждый из которых будет отвечать за свою доменную область.

В качестве инструмента для разработки серверной части выбран фреймворк Spring с его основной концепцией MVC (Model-View-Controller) [23]. Основное преимущество паттерна MVC – разделение логики приложения на три слоя, что делает код гибким, тестируемым и удобным в поддержке. В данном контексте Model – слой работы с данными, View – представление этих данных, а Controller – обработка входящих запросов. Паттерн позволяет разрабатывать и тестировать слои отдельно, а также дополнять их функционалом без изменения других слоёв.

Серверное приложение будет предоставлять клиентам REST API, т.к. это одно из самых распространённых, легко поддерживаемых и удобных решений. С учётом описанных подходов, разработана архитектура серверной части веб-приложения и построена схема компонентов, представленная на рисунке 2.8.

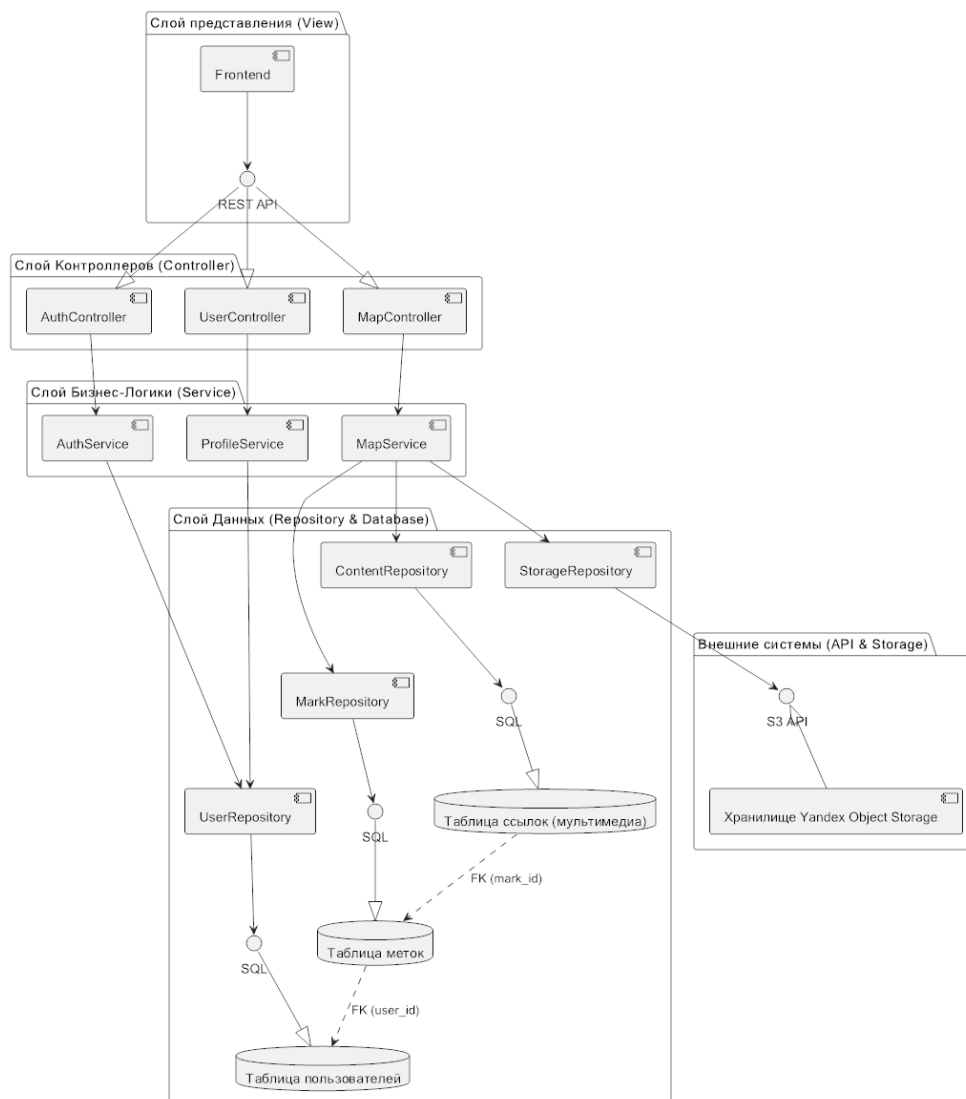


Рисунок 2.8 – Диаграмма компонентов серверной части

Схема разделяет слои на модель, представление и контроллер, тем самым отражая паттерн MVC. Также на диаграмме показано отделение слоя бизнес логики, данных, работы с внешними системами и обработка входящих запросов.

2.4 Проектирование интеграции клиентской и серверной частей

Взаимодействие клиентской и серверной частей будет реализовано с помощью протокола HTTP. Серверная часть будет предоставлять для клиентов REST API. Данный вариант имеет множество преимуществ, такие как кроссплатформенность, использование стандартизированных методов и поддержка кэширования. Обобщённая компонентная диаграмма, которая отражает целостное взаимодействие компонентов, показана на рисунке 2.9.

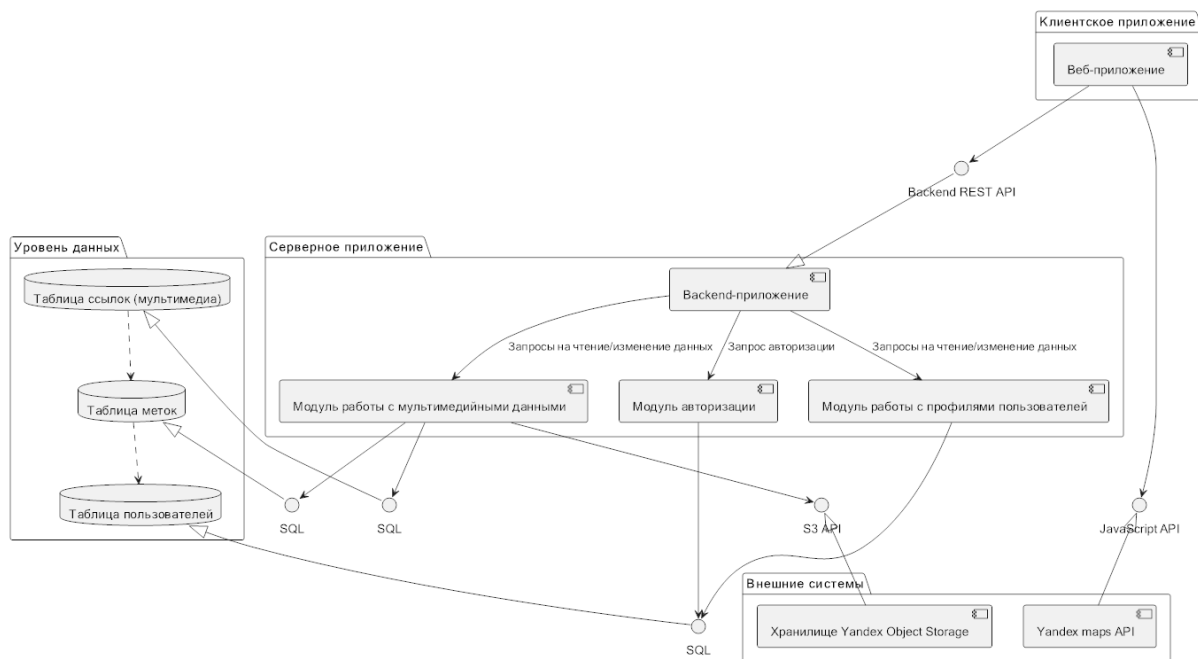


Рисунок 2.9 – Диаграмма компонентов веб-приложения

Схема отражает верхний уровень и показывает взаимодействие клиентской и серверной частей, работу с данными и с внешними системами.

Стоит также уделить внимание вопросам отказоустойчивости и мониторинга. При проектировании важно учитывать возможные сбои, перегрузки и риски потери данных. Для этого система должна включать механизмы отказоустойчивости, автоматического восстановления и

мониторинга состояния компонентов. Определим возможные риски и механизмы их разрешения:

- Перегруз системы при большом количестве одновременных запросов
- поможет горизонтальное автоматическое масштабирование экземпляров серверного приложения с помощью Kubernetes. Масштабирование способно предотвратить деградацию системы в случае увеличения нагрузки;
- Отказ отдельных модулей или экземпляров серверного приложения – поможет мониторинг с Prometheus и Grafana и автоматическое восстановление через Kubernetes. Spring Boot Actuator для встроенного мониторинга состояния;
- Потеря данных в рантайме из-за сбоев – предотвратить проблему сможет резервное копирование и использование облачного хранилища для мультимедийных данных. Хранилище Yandex Object Storage поддерживает версии файлов и обеспечивает отказоустойчивую инфраструктуру;
- Долгое время ответа сервера – решением может стать кэширование данных и оптимизация SQL-запросов, что можно удобно сделать с помощью JOOQ.

Также проблемы с производительностью может решить балансировка нагрузки через Kubernetes с помощью Ingress. Это позволит равномерно распределить запросы между экземплярами приложения.

2.5 Проектирование схемы базы данных

Для хранения данных приложения понадобится несколько таблиц в реляционной базе данных. В качестве СУБД выбрано решение PostgreSQL и S3 хранилище Yandex Object Storage для размещения мультимедийных данных. Стоит отметить, что некоторые таблицы не были представлены на компонентной диаграмме, её содержание направлено на основные элементы. Полная схема базы данных показана на рисунке 2.10.

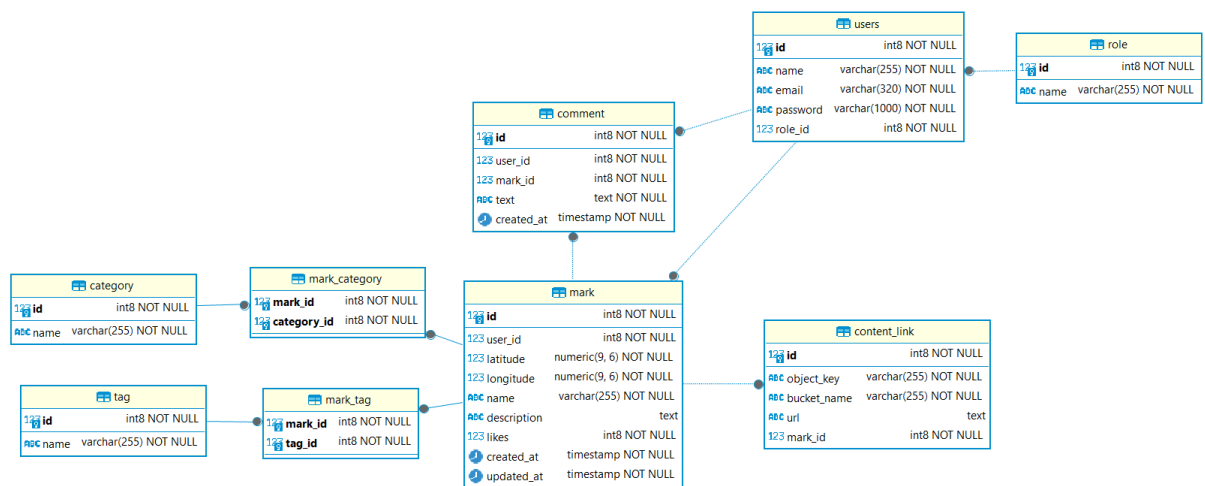


Рисунок 2.10 – Диаграмма базы данных приложения

Для хранения информации о пользователях понадобится две таблицы – с персональными данными и с информацией о ролях в системе. На основе таблицы ролей определяются возможности пользователя. Например, обычный пользователь не имеет полного доступа к приложению и не может просматривать всю информацию о других пользователей, в то время как на администратора данные ограничения не накладываются.

Таблица с метками содержит основную географическую информацию, описание и количество лайков, также выстроено отношение с таблицей комментариев. Мультимедийный контент сохраняется в облачное хранилище, а информация о сохранённом объекте содержится в таблице content_link. Также созданы таблицы с тегами и категориями, которые связаны с таблицей меток отношением «многие-ко-многим» для предотвращения дублирования данных и обеспечения нормализованной формы БД [24].

2.6 Вывод ко второму разделу

В данном разделе практики создана функциональная модель системы с помощью нотации IDEF0, описаны бизнес-процессы приложения. Также спроектирована архитектура веб-приложения и архитектуры клиентской и серверной частей, определены паттерны для реализации частей приложения. Для проектирования использовались диаграммы компонентов, отображающие основные элементы приложения. Затем спроектирована схема базы данных, определены необходимые ограничения и связи.

3. Технологический раздел

3.1 Разработка модуля аутентификации пользователей

Для практической реализации в рамках практики выбран модуль аутентификации пользователей. Создано серверное приложение с начальной конфигурацией и подключена зависимость Spring Security [11]. Назначение модуля заключается в предоставлении функционала регистрации и проверки подлинности пользователей при входе в приложение. Аутентификация основана на JWT-токенах, которые генерируются серверной частью и далее передаются клиентской для сохранения и отправки при дальнейших запросах.

Модуль взаимодействует с базой данных PostgreSQL для сохранения информации о пользователях и её валидации при попытках входа в систему. Для обеспечения отказоустойчивости и масштабируемости, модуль будет разворачиваться в Kubernetes кластере, для чего создана соответствующая конфигурация. Также в модуле реализованы механизмы Health Check с помощью Spring Actuator и повторные запросы.

Рассмотрим базовую конфигурацию для Spring Security, реализованную в приложении (листинг 3.1).

Листинг 3.1 – Конфигурация Spring Security

```
@Configuration
@EnableWebSecurity
class SecurityConfig(
    private val jwtFilter: JwtAuthenticationFilter
) {

    @Bean
    fun securityFilterChain(http: HttpSecurity): SecurityFilterChain {
        return http
            .csrf { it.disable() }
            .authorizeHttpRequests {
                it
                    .requestMatchers("/auth/**").permitAll()
                    .anyRequest().authenticated()
            }
            .sessionManagement {
                it.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
            }
    }
}
```

Продолжение листинга 3.1

```
        }
        .addFilterBefore(jwtFilter,
UsernamePasswordAuthenticationFilter::class.java)
        .build()
    }

    @Bean
    fun passwordEncoder(): PasswordEncoder = BCryptPasswordEncoder()
}
```

Для клиентских запросов реализован REST API с двумя точками доступа для регистрации и аутентификации:

- POST /auth/registration – регистрация пользователя;
- POST /auth/login – аутентификация пользователя.

Код контроллера с реализацией точек доступа показан на листинге 3.2.

Листинг 3.2 – Код контроллера аутентификации и регистрации

```
@RestController
@RequestMapping("/auth")
class AuthController(
    private val authService: AuthService
) {

    @PostMapping("/registration")
    fun registerUser(@RequestBody request: RegisterRequest):
ResponseEntity<String> {
        authService.register(request)
        return ResponseEntity.ok("Пользователь успешно зарегистрирован")
    }

    @PostMapping("/login")
    fun loginUser(@RequestBody request: AuthRequest):
ResponseEntity<AuthResponse> {
        val token = authService.authenticate(request)
        return ResponseEntity.ok(token)
    }
}
```

Для развёртывания приложения создана конфигурация Kubernetes, которая состоит из нескольких файлов. Каждый из них отвечает за свою часть

– за описание сервисов, деплоиментов, ключей конфигурации для самого приложения и для базы данных. Приведём основу конфигурации, представленную деплоиментом серверного приложения (листинг 3.3).

Листинг 3.3 – Конфигурация деплоимента модуля

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: photo-map-backend
spec:
  replicas: 3
  selector:
    matchLabels:
      app: photo-map-backend
  template:
    metadata:
      labels:
        app: photo-map-backend
    spec:
      containers:
        - name: photo-map-backend
          image: soneech/photo-map-backend:latest
          ports:
            - containerPort: 8080
          env:
            - name: SPRING_DATASOURCE_URL
              value: jdbc:postgresql://postgres:5432/photo-map-db
            - name: SPRING_DATASOURCE_USERNAME
              valueFrom:
                configMapKeyRef:
                  name: app-config
                  key: POSTGRES_USER
            - name: SPRING_DATASOURCE_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: postgres-secret
                  key: POSTGRES_PASSWORD
```

YAML-файл разворачивает 3 экземпляра Spring Boot приложения (реплика = 3) в Kubernetes-кластере. Приложение запускается в контейнере из Docker-образа и автоматически получает доступ к базе данных PostgreSQL через переменные окружения.

3.2 Вывод к третьему разделу

В результате выполнения раздела реализован модуль аутентификации пользователей, обеспечивающий безопасную обработку учетных данных и выдачу JWT-токенов. Также была разработана базовая конфигурация для развертывания приложения и базы данных в Kubernetes, включающая переменные окружения, секреты и сетевые сервисы. Это обеспечивает гибкость, масштабируемость и устойчивость приложения при его работе в контейнерной среде. Подключен Spring Actuator для использования механизмов Health Check, а также реализованы повторные запросы.

ЗАКЛЮЧЕНИЕ

В результате выполнения проектной практики проведён анализ предметной области, в ходе которого подтверждена актуальность и востребованность выбранной темы, выявлены недостатки существующих решений. Проведён сравнительный анализ сервисов, работающих с геотегированным контентом, что позволило определить ключевые требования к создаваемому приложению.

Сформулированы функциональные и нефункциональные требования к веб-приложению, определены основные бизнес-процессы, а также выбраны оптимальные технологии для разработки клиентской и серверной частей.

В рамках проектирования информационной системы разработаны функциональные схемы в нотации IDEF0 и диаграммы UML, что позволило наглядно представить структуру приложения. Спроектирована архитектура системы, включающая архитектуру клиентской и серверной частей, а также схема базы данных с необходимыми ограничениями и связями. Также реализован модуль аутентификации пользователей.

Полученные результаты полностью соответствуют поставленным задачам и позволяют приступить к непосредственной реализации веб-приложения, обеспечивающего обмен геотегированным мультимедийным контентом с интерактивной картой.

СПИСОК ИНФОРМАЦИОННЫХ ИСТОЧНИКОВ

1. Понятие геотегинга [Электронный ресурс]. — URL: <https://ru.wikipedia.org/wiki/%D0%93%D0%B5%D0%BE%D1%82%D0%B5%D0%B3%D0%B8%D0%BD%D0%B3> (дата обращения 20.02.2025).
2. Live Journal. Заккрытие сервиса Яндекс.Фотки, негативный отзыв [Электронный ресурс]. — URL: <https://nickfw.ru/176879.html> (дата обращения 25.02.2025).
3. Картографические API, обзор и сравнение [Электронный ресурс]. — URL: <https://novainfo.ru/article/13853> (дата обращения 26.02.2025).
4. ГОСТ 7.32-2017. Отчёт о научно-исследовательской работе. Структура и правила оформления [Электронный ресурс]. — URL: <https://docs.cntd.ru/document/1200157208> (дата обращения 28.02.2025).
5. Panoramio, история и особенности [Электронный ресурс]. — URL: <https://ru.wikipedia.org/wiki/Panoramio> (дата обращения 28.02.2025).
6. Habr. Panoramio не умер [Электронный ресурс]. — URL: <https://habr.com/ru/articles/381417/> (дата обращения 28.02.2025).
7. Яндекс.Фотки, сервис для хранения и обмена фотографиями [Электронный ресурс]. — URL: <https://ru.wikipedia.org/wiki/%D0%AF%D0%BD%D0%B4%D0%B5%D0%BA%D1%81.%D0%A4%D0%BE%D1%82%D0%BA%D0%B8> (дата обращения 03.03.2025).
8. Google Earth: виртуальный глобус и географическая информация [Электронный ресурс]. — URL: <https://earth.google.com/web/> (дата обращения 06.03.2025).
9. Wikimapia, краудсорсинговая карта мира [Электронный ресурс]. — URL: <https://wikimapia.org/#lang=ru&lat=56.851900&lon=60.612200&z=12&m=w> (дата обращения 06.03.2025).
10. Spring Framework и Spring Boot, инструменты для разработки на Java [Электронный ресурс]. — URL: <https://blog.doubleslash.de/software->

technologien/vergleich-spring-boot-vs-micronaut-vs-quarkus?utm_source=chatgpt.com (дата обращения 13.03.2025).

11. Spring Security, обеспечение безопасности в приложениях [Электронный ресурс]. — URL: <https://spring.io/projects/spring-security> (дата обращения 13.03.2025).

12. Сравнение Java, Kotlin, Groovy и Scala [Электронный ресурс]. — URL: <https://stackshare.io/stackups/groovy-vs-kotlin-vs-scala> (дата обращения 13.03.2025).

13. Сравнение MySQL, PostgreSQL и MongoDB [Электронный ресурс]. — URL: <https://www.geeksforgeeks.org/comparing-mysql-postgresql-and-mongodb/> (дата обращения 13.03.2025).

14. Сравнение Yandex Object Storage, Amazon S3 и Google Cloud Storage [Электронный ресурс]. — URL: <https://yandex.cloud/en/docs/overview/platform-comparison/> (дата обращения 13.03.2025).

15. Сравнение JPA, JOOQ и JDBC Template [Электронный ресурс]. — URL: <https://www.jooq.org/> (дата обращения 13.03.2025).

16. Сравнение Liquibase и Flyway: инструмент для управления изменениями в базе данных [Электронный ресурс]. — URL: <https://www.baeldung.com/liquibase-vs-flyway> (дата обращения 13.03.2025).

17. Сравнение Vue.js, React, Angular [Электронный ресурс]. — URL: <https://habr.com/ru/articles/831726/> (дата обращения 13.03.2025).

18. Yandex Maps API, инструменты для работы с картами от Яндекс [Электронный ресурс]. — URL: <https://yandex.ru/dev/jsapi-v2-1/doc/ru/> (дата обращения 13.03.2025).

19. Kubernetes, система оркестрации контейнеров [Электронный ресурс]. — URL: <https://kubernetes.io> (дата обращения 13.03.2025).

20. Prometheus, система мониторинга и алертинга [Электронный ресурс]. — URL: <https://prometheus.io/> (дата обращения 13.03.2025).

21. Grafana, платформа для визуализации и аналитики данных [Электронный ресурс]. — URL: <https://grafana.com/> (дата обращения 13.03.2025).

22. Vue.js: руководство по MVVM и компонентно-ориентированному подходу [Электронный ресурс]. — URL: <https://optimizely.github.io/vuejs.org/guide/> (дата обращения 13.03.2025).

23. Spring MVC, модель представления и контроллера в Spring [Электронный ресурс]. — URL: <https://docs.spring.io/spring-framework/docs/current/reference/html/web.html> (дата обращения 13.03.2025).

24. Habr. Нормализация базы данных [Электронный ресурс]. — URL: <https://habr.com/ru/articles/254773/> (дата обращения 15.03.2025).